Attaching nixnan to a Quantized 7B Medical Dialog Agent: Real Exceptions Found, and a Reproducible Instrumentation Crash

1 Context

`simple_medical_agent.py` (in a separate, private project directory, `~/repos/newmed` – not part of this repository, since it depends on a 14 GB local model checkpoint and its own `venv/`) is a small, standalone medical-dialog chat script built on **BioMistral-7B** (`BioMistral/BioMistral-7B`), loaded in 4-bit quantization via `bitsandbytes` (`nf4`, double quantization, FP16 compute dtype). It is otherwise unrelated to the other `SC26-Tutorial` case studies except that it uses this repository's own `nixnan` instrumentation library (`external/nixnan/nixnan.so`) to look for floating-point exceptions inside the model's own CUDA kernels. This report and the logs behind it live here, in `biomistral-test/`, since they document a real `nixnan` run even though the target program itself does not live in this repository.

The goal of this exercise: feed the agent a single symptom, `"dizzy"`, attach `nixnan` with `SAMPLING=512`, and let generation run to completion while collecting exceptions.

Headline result: this did not fully succeed. Every attempt crashed with a segmentation fault before `nixnan` printed its own final aggregate report or the model printed its answer. This document reports what was found anyway (substantial, real floating-point exception activity) and treats the crash itself as a finding, not a footnote – a real, reproducible robustness limit of NVBit-based instrumentation on a model this large and architecturally complex, in contrast to every case study in the sibling `SC26-Tutorial` project, all of which completed cleanly under the same tool.

2 Setup

- **Model:** `BioMistral/BioMistral-7B`, 4-bit NF4 quantization (`bitsandbytes`), FP16 compute dtype, loaded via `transformers`' `device_map="auto"`.
- **Input:** the single symptom `"dizzy"` (piped via stdin, followed by `quit`/`n` to exit cleanly), generating up to `max_new_tokens=300` with temperature 0.7, top-p 0.9, top-k 50.
- **Instrumentation:** `LD_PRELOAD` pointed at this session's `nixnan.so` (built from `parfloat/nixnan`, branch `print_histo_instrn` – the same build used throughout `SC26-Tutorial`), `SAMPLING=512` (samples 1 in every 512 repeat launches of an already-instrumented kernel; the *first* launch of every distinct kernel is always instrumented, since sampling only gates the runtime enable/disable toggle, not the one-time SASS rewrite – see `SC26-Tutorial/autonixnan-doc.tex`, §2.2), exception checking on by default.
- **Environment:** torch 2.6.0+cu124, transformers 4.57.1, bitsandbytes 0.48.1, accelerate 1.10.1, in this directory's own `venv/`.

3 Attempts

Four attempts were made. Each subsequent one added the previously-observed crash-adjacent kernel to a `FUNCTION_BLACKLIST` file (`Logs/function_blacklist_final.txt` holds the final, 2-entry version) to see whether excluding it let the run progress further.

Att.	Blacklist	Lines	Kernels touched	Last kernel launch logged before the crash
1	none	1581	45	cutlass fused FP16 GEMM+ReLU tensor-core kernel (full name: footnote [1])
2	none (reproducibility check)	1570	45	identical to attempt 1
3	the GEMM+ReLU kernel above	1664	91	`CatArrayBatchedCopy_aligned16_contig` (KV-cache concatenation; full name: footnote [2])
4	both of the above	1683	91	same `CatArrayBatchedCopy` kernel again (see §3.1)

All four crashes were `SIGSEGV` ("Segmentation fault (core dumped)"), reported by the shell after the piped-stdin input had already been fully consumed (the `printf 'dizzy\nquit\nn\n'` side of the pipe always finished normally).

Attempts 1 and 2 are the same command run twice, to check whether the crash was a one-off race or a genuinely reproducible fault: it reproduced almost exactly (45 kernels touched both times, near-identical exception counts – see Table 2 – and the identical last-logged kernel), confirming this is deterministic, not transient.

Each blacklist round did make real forward progress: attempt 3 more than doubled the number of distinct kernels successfully touched (45 → 91) and got well past the point where attempts 1–2 died. Attempt 4 progressed further still within the log (558 vs. 530 distinct NaN sites, more sampling-related kernels touched – `ArgMaxOps`, an exponential-distribution kernel used in nucleus sampling), but still crashed.

3.1 A caveat on "last kernel logged"

`nixnan` prints `running kernel [...]` once per distinct kernel, at launch-interception time, independent of whether that kernel is actually whitelisted/blacklisted for instrumentation. This means the last line printed before a crash is not a reliable pointer to the actual crash site: if the true crash happens during NVBit's one-time SASS analysis of the *next* distinct kernel, that kernel may crash before it ever gets to print its own "running kernel" line. This is the likely explanation for why attempt 4's last-logged kernel is the same `CatArrayBatchedCopy` kernel as attempt 3's, even

[1] `cutlass::Kernel2<cutlass_80_tensorop_f16_s16816gemm_relu_f16_256x64_32x4_tn_align8>`

[2] `CatArrayBatchedCopy_aligned16_contig<OpaqueType<4u>, unsigned int, 3, 128, 1>` (template args abbreviated)

though that specific kernel had already been blacklisted going into attempt 4 – the log line reflects a normal (blacklisted, uninstrumented, fast) launch of that kernel, not necessarily the fault. Pinpointing the true crash site conclusively would require a proper stack trace; this was attempted (via Ubuntu's `apport` crash report) but abandoned, since the resulting core dump was **57 GB** – a 7B-parameter model's GPU-mapped memory is counted in the crashing process's own address space – making it impractical to load into `gdb` in this environment.

4 Exceptions found

Despite never reaching completion, every attempt captured substantial, genuine floating-point exception activity, summed across all instrumented kernels in that run:

Attempt	NaN	-Infinity	Subnormal	NaN+Subnormal	Other combos
1	522	214	301	138	1
2	518	214	296	137	0
3	530	304	229	136	11
4	558	304	229	136	2

Table 2: Distinct exception sites (instruction + operand combinations), by category, per attempt. "Other combos" groups the rare `NaN,Infinity`, `NaN,-Infinity,subnormal` and `NaN,Infinity,subnormal` joint labels. These are counts of *distinct* sites, not occurrence counts – each site typically recurs many times across the model's 32 transformer layers and multiple attention heads.

Two distinct exception patterns stand out, both mechanisms already recognized from the `SC26-Tutorial` case studies rather than novel ones:

- **Subnormal-heavy FP16 tensor-core GEMMs.** The bulk of the `subnormal` sites are `HMMA.16816.F32` tensor-core matrix-multiply-accumulate instructions inside `cutlass::Kernel2` template instantiations – i.e., the dequantized 4-bit weights, multiplied through FP16 tensor cores, routinely produce subnormal intermediate products. This is consistent with 4-bit quantization's narrow dynamic range interacting with FP16's already-narrow exponent range.
- **`-Infinity` from causal-mask softmax.** A large share of the `-infinity` and `NaN,-infinity` sites are `FADD`/`FFMA`/`MUFU.EX2` sequences matching the same `exp(x) = exp2((x - max)` × `log2(e))` Kahan-style decomposition already found to interact badly with causal-masked `-inf` values in `SC26-Tutorial`'s `Karpathy-NGram` case study (see that project's `All-Runs.md`). The mechanism is the same: masked attention positions carry a literal `-inf` into the softmax kernel, and intermediate arithmetic on it produces detectable, instruction-level `-Infinity`/`NaN` values – resolved to a harmless `0` by the time softmax finishes, in every previously-observed case.

5 The crash as a finding

Every other `nixnan` case study built in `SC26-Tutorial` – spanning custom CUDA kernels, HeCBench ports, a small PyTorch/cuSOLVER routine, a toy GPT, and a recurrent network – completed to a clean final report, even when that required working around a large one-time NVBit enumeration cost (documented at length for the PyTorch-backed studies). BioMistral-7B is qualitatively different: it

is dramatically larger (7 billion parameters vs. a few thousand to a few million in every other study here), combines 4-bit quantization (`bitsandbytes`), FlashAttention, and `cutlass` fused-epilogue tensor-core kernels in one pipeline, and reproducibly crashes NVBit's instrumentation machinery partway through a single forward pass, regardless of `SAMPLING` (since sampling does not affect the one-time SASS rewrite that is presumably where the fault lies).

This is worth stating plainly as a limitation of the tool at its current state, on workloads at this scale and complexity – not a failure specific to this particular medical agent or to the `"dizzy"` input. The progressive-blacklisting results (§3) suggest the fragility is not confined to one single kernel, but recurs across at least two distinct, unrelated kernel families (a fused tensor-core GEMM epilogue, and a tensor concatenation kernel used in KV-cache management), with the true trigger of the fourth crash not conclusively identified (§3.1).

6 What was not tried

Given time constraints, the following mitigations were considered but not attempted, and would be reasonable next steps:

- Continuing the blacklist-and-retry loop further (each round has made genuine progress; it is unclear how many more distinct crash sites remain).
- Running without 4-bit quantization (`use_4bit=False`), to check whether `bitsandbytes`' own custom kernels (`kQuantizeBlockwise`/`kDequantizeBlockwise`) are implicated, independent of the `cutlass`/`CatArray` kernels already isolated.
- Capping `max_new_tokens` much lower (e.g. 8–16) to shrink the number of distinct decode-step kernel launches per run, trading completeness of the response for a better chance of reaching `nixnan`'s final report.
- Splitting the run `iter-autonixnan`-style (one kernel whitelisted per fresh process) – impractical for a single coherent multi-token `generate()` call as used here, but could still be adapted to at least identify every crash-triggering kernel systematically, one process at a time.
- Obtaining a proper stack trace via a more targeted core-dump strategy (e.g. attaching `gdb` live with a breakpoint, rather than post-mortem on a 57 GB dump).

7 Reproducing this

This requires `~/repos/newmed` (the private project directory holding `simple_medical_agent.py`, its `venv/`, and the local BioMistral-7B checkpoint) alongside a checkout of this repository, for `external/nixnan/nixnan.so`:

```
cd ~/repos/newmed
source venv/bin/activate
NIXNAN_LIB=~/repos/SC26-Tutorial/external/nixnan/nixnan.so
BLACKLIST=~/repos/SC26-Tutorial/biomistral-test/Logs/function_blacklist_final.txt

# Attempt 1 (no blacklist)
printf 'dizzy\nquit\nn\n' | \
  LD_PRELOAD=$NIXNAN_LIB SAMPLING=512 python3 -u simple_medical_agent.py \
  > attempt1-sampling512.log 2>&1
```

```
# Attempt 4 (both crash-adjacent kernels blacklisted)
printf 'dizzy\nquit\nn\n' | \
  LD_PRELOAD=$NIXNAN_LIB SAMPLING=512 FUNCTION_BLACKLIST=$BLACKLIST \
  python3 -u simple_medical_agent.py \
  > attempt4-blacklist-relu-gemm-and-catarray.log 2>&1
```

All four logs from the original run are in `Logs/` alongside this report.